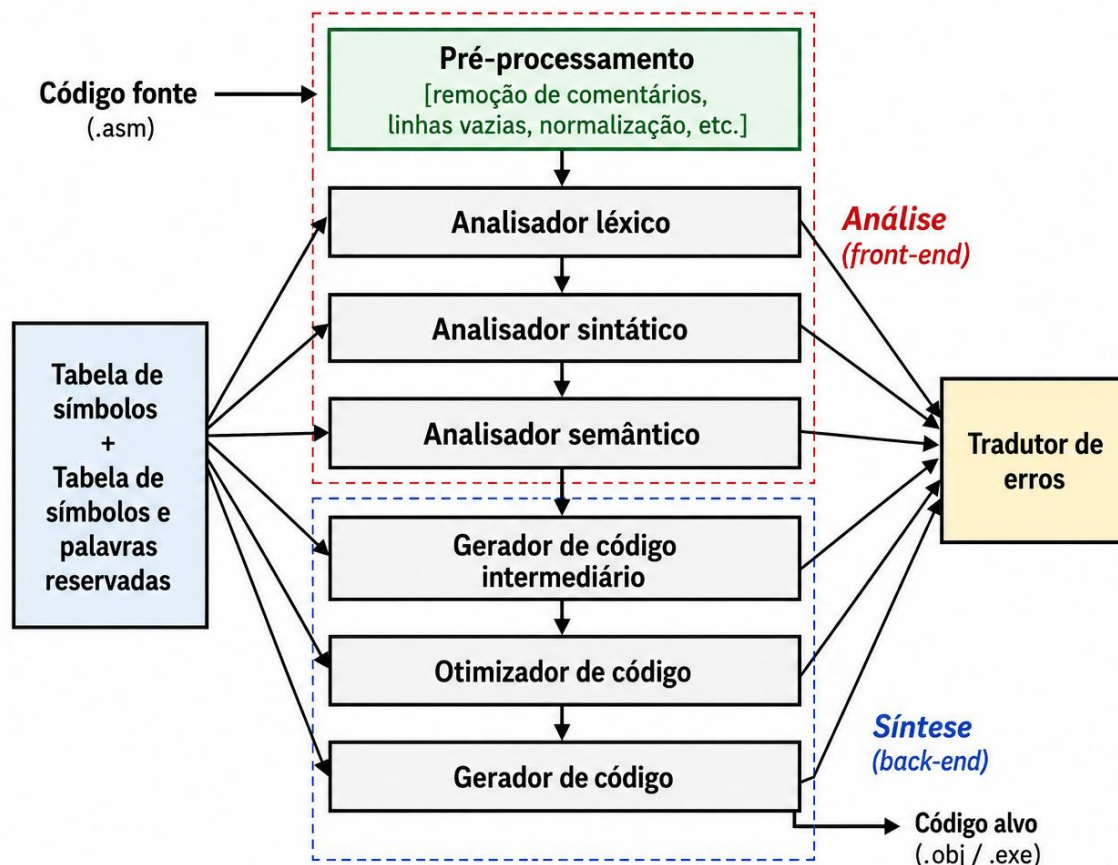


O PRÉ-PROCESSADOR

1. Definições:

O pré-processador é a etapa inicial do processamento de um programa, responsável por preparar o código-fonte antes que ele seja submetido às etapas formais de análise. Em um programa escrito em Assembly MIPS, o pré-processamento pode realizar tarefas como remover comentários e linhas vazias, eliminar espaços e tabulações desnecessários e normalizar a representação das instruções. Seu objetivo é produzir uma versão mais limpa e padronizada do código, preservando as informações relevantes para as etapas seguintes. Dessa forma, o pré-processador simplifica a entrada que será fornecida ao analisador léxico, facilitando a identificação dos tokens que representam instruções, registradores, rótulos, diretivas, valores numéricos e outros elementos da linguagem.



Etapas do projeto Compilador μ -Assembly

Como parte do projeto da disciplina de **Linguagens Formais, Autômatos e Compiladores**, você deverá implementar pré-processador para a linguagem **μ -Assembly** em um módulo independente, com casos de teste específicos para comentários, strings, espaços, linhas vazias e rótulos.

2. Funcionamento:

Nesta etapa do projeto, deverá ser desenvolvido o **pré-processador** de um assembler para um subconjunto da linguagem Assembly MIPS (que chamaremos de **μ-Assembly**), tomando como referência a sintaxe utilizada pelo simulador MARS.

Definição

O pré-processamento corresponde à primeira etapa do tradutor e tem como objetivo preparar e normalizar o código-fonte, eliminando elementos que não são relevantes para as etapas posteriores, sem modificar o significado do programa. O programa deverá receber como entrada um arquivo contendo código Assembly MIPS, com extensão **.asm**, e produzir como saída uma representação normalizada desse mesmo programa, que posteriormente será utilizada pelo analisador léxico.

2.1. Entrada

O pré-processador deverá ler um arquivo-texto contendo um programa escrito em Assembly MIPS.

Exemplo:

```
# Programa de exemplo

.data
mensagem: .asciiz "Resultado: "    # mensagem exibida
valor:    .word 10

.text
main:
    li $t0, 5      # primeiro valor
    li $t1, 10     # segundo valor

    add $t2, $t0, $t1
```

Esse programa, escrito em **μ-Assembly**, tem como principal finalidade realizar a soma de dois valores e armazenar o resultado em um registrador. Inicialmente, a diretiva **.data** identifica a seção destinada à declaração dos dados que serão armazenados na memória. Nessa seção, o rótulo **mensagem** identifica a string "Resultado: ", declarada por meio da diretiva **.asciiz**, enquanto o rótulo **valor** identifica uma palavra de memória inicializada com o número 10 pela diretiva **.word**.

Em seguida, a diretiva **.text** determina o início da seção que contém as instruções executáveis do programa, enquanto o rótulo **main** identifica o ponto inicial do código apresentado. A pseudoinstrução **li \$t0, 5** carrega o valor 5 no registrador \$t0, e **li \$t1, 10** armazena o valor 10 no registrador \$t1. Posteriormente, a instrução **add \$t2, \$t0, \$t1** soma os valores armazenados em \$t0 e \$t1, colocando o resultado no registrador \$t2. Dessa forma, ao final da execução, \$t2 contém o valor 15.

Embora sejam declarados a mensagem "Resultado: " e o valor 10 na seção de dados, esses elementos não são utilizados pelas instruções apresentadas. Além disso, o resultado da soma permanece armazenado no

registrador \$t2 e não é exibido no console, pois o programa não contém chamadas de sistema (syscall) destinadas à impressão. Assim, o exemplo permite observar de maneira simples a organização de um programa MIPS em seções de dados e código, bem como o uso de registradores e de uma instrução aritmética básica.

2.2. Remoção de comentários

No contexto de programação e, especificamente, do Assembly MIPS, um comentário é um texto inserido no código-fonte com a finalidade de explicar ou documentar o programa. Ele serve para facilitar a leitura e a compreensão do código, mas não representa uma instrução a ser executada pelo processador.

No MIPS/MARS, os comentários são normalmente indicados pelo caractere `#`. Tudo o que aparece depois desse caractere, até o final da linha, é considerado comentário e deve ser removido pelo pré-processador. Assim, a instrução

```
add $t0, $t1, $t2      # realiza a soma
```

deverá resultar em:

```
add $t0, $t1, $t2
```

Entretanto, o caractere `#` não deverá ser interpretado como início de comentário quando estiver no interior de uma cadeia de caracteres.

Por exemplo:

```
msg: .asciiz "Valor # informado"
```

deverá permanecer inalterado.

2.3. Remoção de linhas vazias

Linhas que, após a remoção de comentários e espaços desnecessários, não possuírem qualquer conteúdo deverão ser eliminadas. Dessa forma, o arquivo produzido pelo pré-processador não deverá conter linhas vazias entre as instruções, diretivas ou rótulos.

2.4. Normalização de espaços e tabulações

O pré-processador deverá substituir tabulações por espaços e eliminar espaços desnecessários no início e no final das linhas. Logo, sequências de vários espaços utilizadas apenas para separação dos elementos também deverão ser normalizadas.

Por exemplo:

```
add      $t0, $t1, $t2
```

deverá ser normalizado para:

```
add $t0, $t1, $t2
```

A normalização não deverá modificar os espaços existentes no interior de cadeias de caracteres (strings). Por exemplo, o pré-processador deve manter todos os espaços presentes na string `msg` indicada a seguir:

```
msg: .ascii "Resultado da operação"
```

2.5. Normalização das quebras de linha

O programa deverá tratar adequadamente arquivos produzidos em diferentes sistemas operacionais, considerando as formas usuais de representação de quebra de linha. Ademais, a saída deverá utilizar uma representação uniforme para separar as linhas do programa.

2.6. Preservação de rótulos

Os **rótulos** são *nomes simbólicos atribuídos a posições do programa ou da memória* e permitem que o programador faça referência a um endereço por meio de um nome, sem precisar conhecer ou escrever diretamente o endereço numérico correspondente.

Por exemplo:

```
.data
mensagem: .ascii "Olá!"
valor:    .word 10

.text
main:
    li $t0, 5
```

Nesse código, temos três rótulos: *mensagem*, que identifica o endereço onde começa a string "Olá!"; *valor*, que identifica o endereço onde está armazenado o número 10; e *main*, que identifica uma posição na seção de código. Esses rótulos, na execução do pré-processador, deverão ser preservados, pois serão necessários posteriormente para a construção da tabela importante, chamada tabela de símbolos.

Observação:

Nesta etapa, não deverá ser verificado se o rótulo foi declarado corretamente nem deverá ser calculado o endereço associado a ele.

2.7. Preservação das diretivas

No contexto de μ -Assembly, uma **diretiva** é uma instrução destinada ao **montador (assembler)**, e não ao processador. Ela orienta o montador sobre como organizar o programa, reservar memória, armazenar dados ou definir determinadas seções do código.

Por exemplo:

```
.data
mensagem: .ascii "Resultado: "
valor:     .word 10

.text
main:
    li $t0, 5
```

Nesse código:

- **.data** indica o início da seção destinada aos **dados**;
- **.ascii** determina que uma sequência de caracteres seja armazenada na memória e terminada com `\0`;
- **.word** determina o armazenamento de um valor inteiro de **32 bits**;
- **.text** indica o início da seção que contém as **instruções do programa**.

A principal diferença é que uma diretiva *não corresponde diretamente a uma instrução executada pela CPU*. Por exemplo, `add $t0, $t1, $t2` representa uma operação que o processador executará; já `.data` apenas fornece uma orientação ao assembler sobre a organização do programa.

No desenvolvimento do projeto, o pré-processador não deverá interpretar semanticamente essas diretivas. Seu reconhecimento e tratamento serão realizados nas etapas posteriores do projeto.

2.8. Preservação de strings

Uma string é uma *sequência de caracteres delimitada por aspas* e utilizada para representar textos em um programa. Esses caracteres podem ser letras, números, espaços, símbolos e sinais de pontuação.

Por exemplo:

```
mensagem: .ascii "Resultado: "
```

Nesse caso, `"Resultado: "` é uma **string**, formada pela sequência de caracteres R, e, s, u, l, t, a, d, o, :, espaço. No **MIPS/MARS**, as strings são armazenadas na memória como uma sequência de bytes, normalmente utilizando a codificação ASCII. Quando usamos a diretiva `.ascii`, o assembler acrescenta automaticamente um **caractere nulo (`\0`) ao final da string**, que serve para indicar onde o texto termina.

No desenvolvimento do projeto, o pré-processador não deverá preservar o conteúdo interno das strings, inclusive quando possuir vários espaços consecutivos, caracteres utilizados normalmente como separadores e o próprio caractere `#`.

2.9. Saída esperada

Considerando como entrada:

```
# Exemplo de programa

.data

msg:    .asciiz "Resultado # obtido"    # mensagem

.text
main:
    li    $t0, 5

    li    $t1, 10    # segundo número
    add   $t2, $t0, $t1
```

uma saída válida do pré-processador seria:

```
.data
msg: .asciiz "Resultado # obtido"
.text
main:
li $t0, 5
li $t1, 10
add $t2, $t0, $t1
```

Observe que comentários, linhas vazias e espaços desnecessários foram removidos, enquanto rótulos, diretivas e o conteúdo da string foram preservados.

2.10. Delimitação desta etapa

O pré-processador deverá realizar exclusivamente operações de preparação e normalização textual.

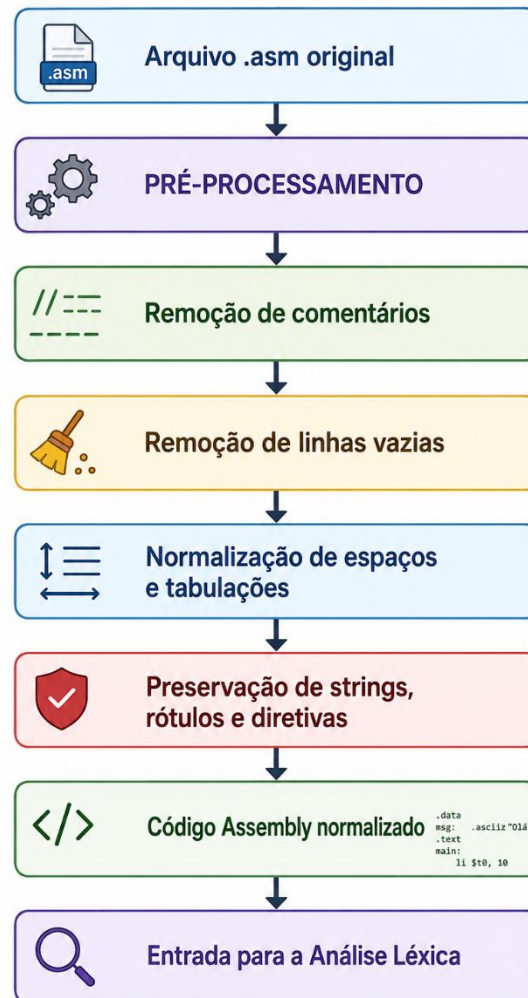
Portanto, não fazem parte desta etapa:

- reconhecimento e classificação de tokens;
- validação da sintaxe das instruções;
- verificação da quantidade de operandos;
- validação dos nomes dos registradores;
- verificação da existência de rótulos;
- construção da tabela de símbolos;
- cálculo de endereços;
- expansão de pseudoinstruções;
- conversão das instruções para código de máquina.

Por exemplo, caso apareça *add \$t0, \$t1*, o pré-processador não deverá acusar erro pela ausência do terceiro operando. Essa responsabilidade será atribuída à análise sintática. Da mesma forma *add \$t0, \$t1, \$t99* não deverá gerar erro nesta etapa apenas porque *\$t99* não corresponde a um registrador MIPS válido. Essa verificação será realizada posteriormente.

2.11. Resultado da etapa

Ao final do pré-processamento, deverá ser possível representar o funcionamento do módulo da seguinte forma:



O código produzido deverá manter o mesmo significado do programa original, modificando apenas aspectos textuais necessários para simplificar e padronizar a entrada das próximas etapas do assembler.

2.12. Requisitos

O código-fonte do **pré-processador** deverá ser desenvolvido em **linguagem C**, a ser compilado e executado em ambiente Windows, e obrigatoriamente modularizado nos seguintes três arquivos:

- **main.c** deverá conter a função `main()` e ser responsável pelo recebimento e validação dos argumentos da linha de comando, pela abertura dos arquivos e pelo acionamento das funcionalidades do pré-processador;
- **preprocessador.c**: deverá conter a implementação das funções responsáveis pelas etapas de pré-processamento do código Assembly;

- **preprocessador.h** : deverá conter os protótipos das funções, constantes, tipos e demais declarações necessárias ao módulo de pré-processamento.

Considerando a organização em main.c, preprocessador.c e preprocessador.h, a compilação pode ser feita com o **GCC** da seguinte forma:

```
gcc main.c preprocessador.c -o main.exe
```

O arquivo preprocessador.h **não precisa ser informado explicitamente** no comando de compilação, pois ele será incluído nos arquivos .c por meio da diretiva *#include "preprocessador.h"*.

Após a compilação, o programa deverá ser executado por meio da **linha de comando**, recebendo obrigatoriamente dois argumentos: o nome do arquivo Assembly de entrada e o nome do arquivo que receberá o resultado do pré-processamento.

A execução deverá seguir o formato:

```
./main.exe <arquivo_entrada.asm> <arquivo_saida.pre>
```

Por exemplo, em ./main.exe teste.asm teste.pre, o programa deverá ler o código Assembly contido em **teste.asm**, realizar todas as operações de pré-processamento especificadas no trabalho e gravar o resultado no arquivo **teste.pre**.

Caso a quantidade de argumentos seja diferente da esperada ou não seja possível abrir algum dos arquivos, o programa deverá apresentar uma **mensagem de erro clara e encerrar sua execução de forma controlada**.